Blushy Architecture Change Comparison

What should change, why it matters, and what improvement to expect

Scope: BLUSHY_MAINAPP Flutter client, Node.js/Express backend, MongoDB, AI integration, WebSockets, uploads, and scheduled jobs.

Executive answer: The current architecture is a reasonable modular monolith foundation. The recommended changes are worthwhile because they mainly reduce security, privacy, data-consistency, and deployment risks without requiring a full rewrite.

How to read the percentages: These are estimated risk reductions or quality improvements based on the identified code patterns. They are not measured benchmark results. Actual performance percentages require load tests and production telemetry.

70-95%	40-75%	30-60%
Security/privacy risk reduction	Reliability improvement	Maintainability improvement

Prepared: August 2026 | Baseline: current source structure and local configuration

1. Highest-Value Changes

Area	Nothing changed	After the change	Estimated improvement
Admin access	A route using optional authentication can accidentally expose private admin data.	Every admin route requires a valid token and an explicit admin role.	**90-100% risk reduction**
AI credentials	A provider key may be returned to the Flutter client.	The permanent key stays on the backend; voice uses an ephemeral token or proxy.	**95-100% risk reduction**
Private routes	Some routes depend on controller checks after optional authentication.	Private routes use mandatory authentication at the route boundary.	**60-90% missed-check risk reduction**
Health-data logs	Full transcription or AI content may appear in logs.	Logs contain safe metadata only: request ID, operation, status, and duration.	**70-95% privacy exposure reduction**
Production JWT	A known fallback secret can be used if deployment configuration is missing.	Production startup fails unless a long random secret is configured.	**90-100% default-secret risk reduction**
Upload safety	Some attachments have broad or inconsistent validation.	Allowed types, file signatures, size limits, quotas, and scanning are enforced.	**60-85% upload abuse reduction**

These changes protect users and the business. They usually have better value than early large-scale performance rewrites.

2. Data Reliability Changes

Area	Nothing changed	After the change	Estimated improvement
Database transactions	`withTransaction()` runs a function but does not create a MongoDB transaction. Multi-document updates can be partial.	Operations that must succeed together use MongoDB sessions and real transactions.	**50-90% partial-write risk reduction**
Index failures	Index creation errors are logged, but the server continues running.	Required-index failures stop startup or make readiness unhealthy.	**30-70% data/query reliability improvement**
User collections	Woman/man collections duplicate repository logic and increase selection mistakes.	A central resolver or unified identity model controls collection selection.	**30-60% role-query defect reduction**
Medical ownership	Every medical read may not be visibly and consistently scoped to the authenticated user.	Every query includes user ownership and isolation tests prove it.	**70-95% cross-user exposure risk reduction**

3. Runtime and Scaling Changes

Area	Nothing changed	After the change	Estimated improvement
Token refresh	An expired access token can force an unexpected logout even though refresh support exists on the server.	A shared client interceptor refreshes once, retries once, and logs out only when refresh fails.	**30-60% fewer auth interruptions**
Shared HTTP client	Services can configure headers, errors, and timeouts differently.	One authenticated client owns base URL, headers, refresh, timeout, and error mapping.	**25-50% less integration maintenance**
WebSockets	Connections are stored in one Node process. Events can disappear when multiple servers run.	Redis Pub/Sub or another shared broker distributes events across instances.	**70-95% multi-instance event reliability**
WebSocket auth	JWT is passed in `/ws?token=...`, which can enter URL logs.	A short-lived ticket or authenticated upgrade avoids exposing the main JWT in the URL.	**60-85% credential exposure reduction**
Scheduled jobs	Schedulers run inside every web process, so multiple instances can duplicate work.	Jobs run in a worker or use a distributed lock.	**70-95% duplicate-job risk reduction**
Redis deployment	In-memory behavior differs from multi-instance behavior.	Production explicitly requires shared Redis for rate limits and events.	**60-90% distributed-state defects reduction**

4. Flutter Maintainability Changes

Area	Nothing changed	After the change	Estimated improvement
Large screens	Dashboard/onboarding widgets mix UI, API calls, calculations, caching, and state changes.	Screens render state; view models/controllers orchestrate; repositories handle data.	**30-60% easier maintenance**
Startup routing	Routing can inspect state while local session loading is still in progress.	An explicit loading state completes before authenticated routing.	**50-80% startup race reduction**
API contract	Both `/auth` and `/api/auth` styles exist, and docs can drift.	One versioned API convention is used by backend, Flutter, tests, and docs.	**40-70% setup/integration confusion reduction**

5. Testing and Delivery Changes

Area	Nothing changed	After the change	Estimated improvement
Backend test command	No standard `npm test` command; tests may depend on a live database.	A repeatable test command uses an isolated test database or temporary MongoDB.	**50-80% higher test confidence**
Authorization tests	Admin, WebSocket, upload, refresh, and privacy boundaries can regress unnoticed.	Anonymous, non-admin, cross-user, expired-token, and invalid-upload cases are automated.	**50-90% regression detection improvement**
Observability	Sensitive logs and incomplete readiness signals make diagnosis harder.	Safe structured logs, health/readiness checks, and request IDs show what failed.	**25-50% faster diagnosis estimate**

6. Overall Before and After

Quality measure	Nothing changed	After the recommended work	Estimated improvement
Security	Good basic middleware, but important boundary inconsistencies remain.	Sensitive routes, secrets, uploads, WebSockets, and admin access have explicit controls.	**70-90%**
Privacy	Health and AI data require more consistent ownership and logging controls.	Data access is user-scoped, logs are minimized, and privacy cases are tested.	**70-95%**
Reliability	Partial writes, token expiry, silent index failures, and duplicate jobs are possible.	Transactions, readiness checks, refresh, and worker coordination cover these cases.	**40-75%**
Scalability	Single-process WebSockets and in-process schedules are suitable mainly for one instance.	Shared events and separate jobs support multiple backend instances.	**60-90%**
Maintainability	The structure is understandable, but large screens/controllers carry too many responsibilities.	Responsibilities are smaller, contracts are consistent, and tests protect boundaries.	**30-60%**

7. Recommended Order

1. **First:** secure admin routes, keep AI keys server-side, protect medical ownership, and remove sensitive logs.

2. **Second:** enforce authentication, validate production secrets, strengthen uploads, and add authorization tests.
3. **Third:** add token refresh, real transactions, reliable index startup, and consistent API paths.
4. **Fourth:** add Redis event fan-out, separate scheduled workers, and refactor large Flutter screens.

> **Decision:** The changes are worth doing. The first group gives the highest value with the least disruption. A full rewrite is not justified by the current findings; targeted improvements preserve the existing Flutter, Express, and MongoDB investment.
